

Code Review — recapitulare-vacanta

Stadiu: Problema 1a rezolvată în două variante (`solutie1` — parcurgere directă, `solutie12` — vector de frecvență). Bravo pentru a doua abordare — frecvența te pregătește direct pentru punctele b) și c).

🔴 Bug-uri critice (rezolvă întâi)

1. `frecventa` — off-by-one la inițializare (`fisier.h:55`)

Bucula de zerorizare merge până la `i < 9999`, deci `f[9999]` rămâne cu gunoi din memorie. Dar `valMinima` scanează până la `i <= 9999`. Dacă gunoiul din `f[9999]` e nenul și vectorul tău nu conține valori mai mici, programul raportează un minim fals.

Fix: folosește aceeași limită peste tot, printr-o constantă:

```
const int MAXV = 10000;

void frecventa(int v[], int n, int f[])
{
    for(int i = 0; i < MAXV; i++)
    {
        f[i] = 0;
    }
    for(int i = 0; i < n; i++)
    {
        f[v[i]]++;
    }
}
```

Și declară vectorul cu aceeași constantă: `int f[MAXV];`

Verifică: rulează de mai multe ori — rezultatul trebuie să fie identic la fiecare rulare, indiferent ce era în memorie.

2. `valMinima` — lipsește `return` pe toate căile (`fisier.h:65-74`)

Dacă tot vectorul `f` e zero (de exemplu `n = 0`), funcția se termină fără `return`. O funcție `int` care nu returnează nimic are **comportament nedefinit** — poate afișa orice.

Fix: returnează o valoare-semnal la final:

```
int valMinima(int f[])
{
    for(int i = 0; i < MAXV; i++)
    {
        if(f[i] != 0)
        {
            return i;
        }
    }
    return -1;
}
```

Verifică: compilează cu `g++ -Wall main.cpp` — warning-ul `control reaches end of non-void function` trebuie să dispară.

3. Numerele negative sparg vectorul de frecvență

Enunțul spune „numere întregi de maxim 4 cifre” — asta include și negativele (ex: -56). `f[v[i]]` cu index negativ înseamnă acces în afara vectorului → crash sau rezultate aiurea.

De decis: ori limitezi explicit problema la numere naturale (scrie asta într-un comentariu la citire), ori deplasezi indexul:

```
f[v[i] + 9999]++; // cu int f[2 * MAXV]; indexul 0 corespunde lui -9999
```

Verifică: pune un număr negativ în `data.txt` și rulează — nu trebuie să crape.

🟡 Issue-uri minore (când ai timp)

1. `v[100]` vs enunț — problema cere maxim **300** de numere. Cu `v[100]` și un `data.txt` mai mare suprascrii memorie. Fix: `int v[300];`.
2. `citireVector` nu verifică fișierul (`fisier.h:9`) — dacă `data.txt` lipsește, `dim` rămâne neinițializat și afișezi gunoi. Adaugă după deschidere:

```

if(!read)
{
    dim = 0;
    return;
}

```

3. **Organizarea headerelor s-a diluat** — convenția era: `fisier.h` = doar citire/afișare (I/O), `solutii.h` = logica de rezolvare. `valMin`, `nrAp`, `frecventa`, `valMinima` sunt logică de rezolvare — mută-le în `solutii.h`.
4. **Spațiile din mesaje** — `"...este"<< mn` afișează lipit („este12”). La fel `<< nr << "ori"` în `solutie1`. Aduagă spațiu în string.
5. **bin/ și obj/ sunt în git** — artefactele de build nu se versionează. Creează `.gitignore` cu:

```

bin/
obj/

```

apoi `git rm -r --cached bin obj`.

6. **Mesajele de commit** — „upd” nu spune nimic peste o săptămână. Formatul recomandat: `feat: rezolvare 1a cu vector de frecventa`.

Pattern-uri corecte (continuă așa)

- `valMin` și `nrAp` sunt curate: inițializare corectă cu `v[0]`, buclă de la 1, `return` la final.
- **Două abordări pentru aceeași problemă** — parcurgere directă vs. frecvență. Exact așa se învață: compari costul și limitele fiecărei metode.
- Include guards (`#ifndef/#define/#endif`) corecte în ambele header.
- Separarea `main` / `solutii` / `fisier` — intenția e bună, doar respectă convenția (vezi  punctul 3).
- Citirea din `data.txt` pentru testare rapidă — schimbi datele fără să retastezi la fiecare rulare.

Pași următori

Pas	Cerință	Sugestie de abordare
1b	Cifra maximă dintre elemente + aparițiile ei	descompune fiecare element în cifre cu <code>%10</code> și <code>/10</code>

Pas	Cerință	Sugestie de abordare
1c	Frecvența minimului între pozițiile pmin și pmax	ai deja frecvența — mai ai nevoie de pozițiile min/max
1d	Toate elementele conțin cifra min sau max?	refolosește descompunerea în cifre de la 1b
1e	Primul minim absolut (peste permutări)	citește exemplul din culegere de 2 ori înainte de cod
1f, 1g	Interschimbări min/max	modifici vectorul, deci afișează-l înainte și după
2	Vector de numere reale	float / double — vectorul de frecvență NU mai merge aici; gândește de ce

Sinteză

Criteriu	Stadiu
Corectitudine 1a	✅ ambele variante, cu 3 bug-uri latente (🔴)
Robustețe (fișier lipsă, n=0, negative)	❌ de tratat
Organizare cod (I/O vs logică)	⚠️ parțial
Igienă git (.gitignore, mesaje)	❌ de configurat
Progres culegere	1a din 1a–g + 2a–d